RUNI – Algorithms – Homework 7

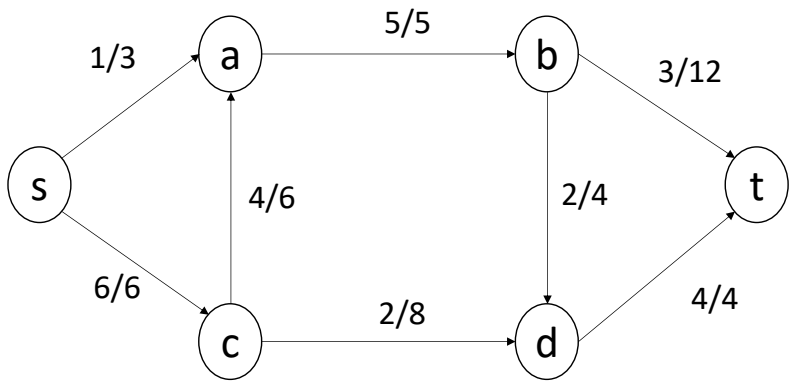
Due date: Sunday 18/01/2026

NOTE: the general homework guidelines published in homework 1 are valid for all homework assignments in this course.

Note that the due-date is more than two weeks away. It is recommended to start early, however, be aware that we will cover the time complexity analysis of max-flow algorithms as well as the material required to solve Problems 4 and 5 only next week. These advanced tasks are marked with ♠.

Problem 1 (20 pts):

The figure presents a flow network with a legal flow function. The given flow is a result of **three iterations** of Ford & Fulkerson algorithm.



Answer the following, no need to justify your answers.

a. The augmenting path the was implemented in the first iteration is

s-c-a-b-d-t

b. Suggest an augmenting path for the next (4-th) iteration, to improve the flow.

s-a-c-d-b-t

c. What is the value of the maximum flow in the network?

9

d. Write all sets of nodes inducing a min-cut in the network.

s (or abcdt), sacd (or bt)

Problem 2 (20 pts):

Complete the following algorithm that decides if a given network $G = (V, E, c)$ has only a single minimum $s - t$ cut. Analyze its time complexity (♠) and justify its correctness.

Algorithm:

1. Run the Edmunds-Karp algorithm to find the maximum flow f. Let N_f be the residual network.

The completion:

2. Build graphs G' and G'' as follows:

    a. $V' = V'' = V$

    b. For all $(u, v) \in E$ such that $f(u, v) < c(u, v)$ then $(u, v) \in E'$ and $(v, u) \in E''$.

3. Run BFS on G' starting from s and find the set of vertices S reachable from s.

4. Run BFS on G'' starting from t and find the set of vertices T reachable from t.

5. If $S \cup T = V$ return TRUE. Otherwise, return FALSE.

Complexity: Total of $O(|E|^2|V|)$, dominated by max flow algorithm.

1. Running Edmunds-Karp algorithm takes $O(|E|^2|V|)$.

2. Construction of the graphs takes $O(|E'| + |V'|) = O(|E| + |V|)$.

3. Running BFS takes $O(|E'| + |V'|) = O(|E| + |V|)$.

Correctness:

By the construction, and correctness of the BFS, we have that step 3 determines the minimum cut in G, which is $(S, V \backslash S)$. This, since S is the set of all vertices reachable from s in N_f.

By the same claims, in step 4, the algorithm finds the set T of all vertices that can reach t in N_f (the vertices that have a directed path to t in N_f).

If $T \neq V \backslash S$ then another legal minimum cut $(V \backslash T, T)$ is found.

If $T = V \backslash S$, then assume towards contradiction there exists another minimum $s - t$ cut, $(A, V \backslash A)$, where $A \neq S$. If $u \in S \backslash A$ then $u \in V \backslash A$ there is a residual path from s to u to t in contradiction. If $u \in A \backslash S$ then $u \in T$ and there is again a residual path from s to u to t in contradiction. Thus $A = S$.

Problem 3 (21 pts)

We are given a network in which all capacities are integers and the max-flow in the network is known to be $F = X > 0$. The capacity of some given edge e is increased. The max-flow in the resulting network is now Y. Prove or give a counter example (for each statement separately):

a. If, before the change, e belong to some min-cut then $X < Y$.

False.

Consider $V = \{s, A, B, t\}, E = \{(s, A), (s, B), (A, t), (B, t)\},$

$c(s, A) = c(s, B) = 10, c(A, t) = c(B, t) = 10, \quad e = (A, t)$

The edge e belongs to a min-cut $\{s, A\}$, which has a capacity 20. However since we have other min-cuts, such as $\{s, B\}$, the increment of e's capacity does not affect the max-flow.

b. If e is upward critical (as defined in the recitation) then $c(e)$ was raised by $Y - X$.

False.

Consider $V = \{s, A, B, t\}, E = \{(s, A), (s, B), (A, t), (B, t)\},$

$c(s, A) = 5, c(s, B) = 15, c(A, t) = 10, c(B, t) = 15, \mathrm{e} = (\mathrm{s}, \mathrm{A})$

We can find that $X = 20$. Clearly, e is forward critical. If we increment e's capacity by 10, Y (the max-flow value) will be 25, since we have min-cut $\{s, A\}$ as well.

c. If $Y = 2X$ then in the original max-flow e was saturated.

True.

Since the increment of $c(e)$ effected the max flow, it effected the capacity of all the min-cuts, meaning e belongs to all the min-cuts in the original network. Moreover, since a max-flow must saturate all outgoing edges of a min-cut, e was saturated.

♠Problem 4 (15 pts)

Consider the maximum matching problem in a bipartite graph. The input is given as an undirected bipartite $G=(V,E)$, where $V=V_1 \cup V_2$, $V_1=\{b_1,\ldots,b_{n1}\}$, $V_2=\{g_1,\ldots,g_{n2}\}$ and $E \subseteq V_1 \times V_2$.

Definition: An edge $(b_i,g_j) \in E$ is **redundant** if the two graphs $G=(V,E)$ and $G'=(V,E-\{(b_i,g_j)\})$ have maximum matchings of the same size.

Suggest an efficient algorithm that returns all the redundant edges in G. Justify the correctness of your algorithm and analyze its time complexity.

1. Build the flow network:
   $V' = V \cup \{s,t\}$.
   $E' = \{(s,v) | v \in V_1\} \cup \{(v,t) | v \in V_2\} \cup (b_i,g_j) \in E$. All edges have capacity 1
2. Run F&F to find a max-flow in this network. Let $f : E \rightarrow \{0,1\}$ be the max-flow function, and let R_f be the last residual network.
3. Go over the edges of G (for example, by going over the adj. lists).

For every edge $(b_i,g_j) \in E$:

If $f((b_i,g_j)=0$ or if $f(b_i,g_j)=1$ and there is a path from b_i to g_j in R_f then output (b_i,g_j).

A path from b_i to g_j can be detected by BFS or DFS.

Correctness: An edge is redundant either if the flow on it in the max-flow is 0, or if the flow on it is 1 and the flow can be re-routed in an alternative way (similar to detection of downward critical edges).

Complexity:

1. Building the network $O(|V'|+|E'|)$. $|V'|=O(|V|)$, $|E'|=O(|V|+|E|)$.
2. Running F&F. Since $f^* \leq \min(|V_1|,|V_2|)$, this takes $O(|E'| \cdot \min(|V_1|,|V_2|))$.
3. If $f(b_i,g_j)=0$ then handling the edge (b_i,g_j) takes O(1).
   If $f(b_i,g_j)=1$ then one run of BFS or DFS is performed. Since $f^* \leq \min(|V_1|,|V_2|)$, and there are f^* such edges, step 3 takes $O(|E'|*\min(|V_1|,|V_2|))$.

All together $O(|E'|*\min(|V_1|,|V_2|)) = O((|V|+|E|) \cdot \min(|V_1|,|V_2|))$.

♠Problem 5 (24 pts)

After a historic year of growth, Nvidia has announced the construction of a new flagship R&D campus in Kiryat Tivon, inspired by the company's California headquarters and expected to become a centerpiece of the northern Israeli tech ecosystem.

To staff the campus, Nvidia decided to launch an accelerated hiring program focused on candidates who have demonstrated strong analytical thinking reflected across multiple evaluation criteria, including outstanding performance in their Algorithms course.

Rather than relying solely on interviews, Nvidia's hiring committee aims to place suitable candidates directly into a precise assignment of roles and teams, while respecting strict internal capacity and qualification constraints.

There are four disjoint sets:

- C – Candidates waiting for an assignment.
- T – development teams at the Tivon campus;
- R – senior responsibility roles (e.g. architecture lead, principal performance engineer).

Each candidate from C may be assigned to at most one team and at most one role.

Each team, and each role, may be assigned at most one new candidate.

For every candidate$c \in C$, Nvidia provides:

- A set $T(c) \subseteq T$ of teams which suit the candidate's overall set of skills;
- A set $R(c) \subseteq R$ of roles that the candidate qualifies for.

a. Performance-Based Flexibility Hiring. In its first hiring attempt, Nvidia allowed flexibility for the set C of candidates. A candidate may fall into one of the following categories:

1. Not Hired.
2. Assigned to exactly one team.
3. Assigned to exactly one role.
4. Assigned to both one team and one role.

The objective of this hiring stage is to **maximize the number of filled positions**.

Design an efficient algorithm that finds an optimal assignment under this hiring policy.

Analyze the time complexity of the algorithm, and justify shortly. No need to prove your algorithm.

Algorithm:

We solve 2 independent one-to-one assignment problems and take their union.

1. Build the directed networks $N_T = (V_T, E_T)$ and $N_R = (V_R, E_R)$ with capacities 1 on all edges.

$$V_T = \{s\} \cup T \cup C \cup \{t\}, V_R = \{s\} \cup R \cup C \cup \{t\}.$$

$$E_T = \{(s, t_i), \forall t_i \in T\} \cup \{(t_i, c_j), \forall t_i \in T(c_j), \forall c_j \in C\} \cup \{(c_i, t) \forall c_i \in C\}$$

$$E_r = \{(s, r_i), \forall r_i \in R\} \cup \{(r_i, c_j), \forall r_i \in R(c_j), \forall c_j \in C\} \cup \{(c_i, t) \forall c_i \in C\}$$

2. Compute a maximum $s - t$ flow in N_T and Compute a maximum $s - t$ flow in N_R.

3. Return the union of team assignments and role assignments from the max flows found.

Complexity:

1. Construction of the graphs in

$O(|V_T| + |E_T|)$ where $|V_T| = |\mathrm{C}| + |\mathrm{T}| + 2, |E_T| = |T| + \sum_{c \in C} \mathrm{T(c)} + |C|$

$O(|V_R| + |E_R|)$ where $|V_R| = |\mathrm{C}| + |\mathrm{R}| + 2, |E_R| = |R| + \sum_{c \in C} \mathrm{R(c)} + |C|$

2. Running Ford-Fulkerson algorithm takes $O(|E_T| \cdot f_T^*) + O(|E_R| \cdot f_R^*)$.

Note that for N_T, $f_T^* \leq \min\{|T|, |C|\}$ and for N_R, $f_R^* \leq \min\{|R|, |C|\}$.

Therefore, **it is faster to run Ford-Fulkerson than to run Edmunds-Karp**.

3. Returning the assignments takes $O(|C|)$.

The total runtime, dominated by FF, is $O((|T| + |R| + |C|) \cdot (\sum_{c \in C} \mathrm{T(c)} + \sum_{c \in C} \mathrm{R(c)} + |T| + |R| + |C|))$.

Note that $\sum_{c \in C} \mathrm{T(c)} \leq |C| \cdot |T|$ and $\sum_{c \in C} \mathrm{R(c)} \leq |C| \cdot |R|$ so total is $O((|T| + |R| + |C|) \cdot (|R| + |T|) \cdot |C|)$.

b. Strict Position and Team Hiring. Following an internal audit, Nvidia concluded that the positions must be filled only by those candidates that are strong enough to justify both their role's technical level and the team's specialty. Thus, under this hiring policy a candidate may only fall into either:

1. Not hired.
2. Assigned simultaneously to one team and one role.

Assignments to only a position or only a role are not permitted.

The objective of this hiring stage again is to **maximize the number of filled positions**.

Design an efficient algorithm that finds an optimal assignment under this hiring policy. Prove the algorithm's correctness (provide formal main claim and prove it). No need to analyze complexity.

In this hiring policy a hired candidate must participate in a triple (t, c, r) with $t \in T(c)$ and $r \in R(c)$. No role, team or candidate can be reused for the triples.

Algorithm:

1. Construct a flow network $N = (V, E)$ with capacity 1 on all edges, and the following:

   a. $V = \{s\} \cup T \cup C_0 \cup C_1 \cup R \cup \{t\}$ where C_0, C_1 are copies of C.

   b. $E = \{(s, t_i), \forall t_i \in T\} \cup \{(t_i, c_j^0), \forall t_i \in T(c_j), \forall c_j \in C\} \cup \{(c_i^0, c_i^1) \forall c_i \in C\}$

   $$\cup \{(c_i^1, r_j), \forall r_j \in T(c_i), \forall c_i \in C\} \cup \{(r_i, t) \forall r_i \in R\}$$

2. Compute maximum $s - t$ flow in N.

3. For each flow path $s \to t \to c_i^0 \to c_i^1 \to r \to t$ output the triple (t, c, r). Explicitly, for each edge $c_i^0 \to c_i^1$ whose capacity is filled in the max flow, locate the corresponding $t \to c_i^0$ and $c_i^1 \to r$ incident that were also filled in the max flow.

Correctness:

The value of the maximum flow equals the maximum number of disjoint triples, and the algorithm returns a matching that hires the maximum possible number of candidates.

Main claim: The network N has a maximum flow of value k if and only if there exists a valid matching consisting of k disjoint triples (t, c, r) where $r \in T(c)$ and $r \in R(c)$.

Proof:

All edges in N have capacity 1. Therefore, a maximum flow of value k decomposes into k edge-disjoint paths from s to t of the form $s \to t \to c_i^0 \to c_i^1 \to r \to t$.

Since (c_i^0, c_i^1) has capacity 1, then each candidate c can appear in at most one such path.

Similarly, since edges incident to team and role vertices also have capacity 1, then each team and each role can appear in at most one such path.

Thus, the k paths correspond to k disjoint triples (t, c, r) forming a valid matching.